



LeetCode 1208: Get Equal Substrings Within Budget

1. Problem Title & Link

Title: LeetCode 1208: Get Equal Substrings Within Budget

Link: <https://leetcode.com/problems/get-equal-substrings-within-budget/>

2. Problem Statement (Short Summary)

You are given:

- String s
- String t
- Integer $maxCost$

Changing:

$s[i] \rightarrow t[i]$

costs:

$abs(ord(s[i]) - ord(t[i]))$

Find the maximum length substring that can be converted within the given budget.

Return the maximum possible length.

3. Examples (Input → Output)

Example 1

Input:

$s = \text{"abcd"}$

$t = \text{"bcdf"}$

$maxCost = 3$

Output:

3

Explanation:

Cost array:

$[1, 1, 1, 2]$

Longest valid window:

$[1, 1, 1]$

Cost = 3

Length = 3

Example 2

Input:

$s = \text{"abcd"}$

$t = \text{"cdef"}$



maxCost = 3

Output:

1

Cost array:

[2,2,2,2]

Only one character can be converted.

Example 3

Input:

s = "abcd"

t = "acde"

maxCost = 0

Output:

1

4. Constraints

- $1 \leq s.length \leq 10^5$
- $s.length == t.length$
- $0 \leq maxCost \leq 10^6$
- Lowercase English letters

Need:

$O(n)$

solution.

5. Core Concept (Pattern / Topic)

Sliding Window

Related Topics:

- Variable Size Window
- Longest Subarray Under Constraint
- Two Pointers

6. Thought Process (Step-by-Step Explanation)

Brute Force

Generate every substring.

For each substring:

Calculate total conversion cost

Complexity:



$O(n^2)$

or worse.

Will TLE.

Key Observation

For every position:

Compute conversion cost.

Example:

$s = \text{"abcd"}$

$t = \text{"bcdf"}$

Costs:

$[1, 1, 1, 2]$

Now the problem becomes:

Find longest subarray

whose sum $\leq \text{maxCost}$

This is a classic Sliding Window problem.

Sliding Window Idea

Maintain:

left

right

currentCost

Expand right pointer.

If:

$\text{currentCost} > \text{maxCost}$

Shrink from left until valid again.

Track maximum window length.

Why It Works

All costs are:

Non-negative

For non-negative arrays,

Sliding Window efficiently finds the longest valid subarray.

7. Visual / Intuition Diagram (ASCII Diagram)

```
Costs:
1  1  1  2
L      R
```



```
Budget:
3
Current Sum:
1+1+1 = 3
Valid.
Expand:
1+1+1+2 = 5
Invalid.
Move left:
1+1+2 = 4
Still invalid.
Move left:
1+2 = 3
Valid.
Maximum length:
3
```

8. Pseudocode (Language Independent)

```
left = 0

currentCost = 0

answer = 0

for right:

    add cost[right]

    while currentCost > maxCost:

        remove cost[left]
        left++

    answer =
        max(answer,
            right-left+1)

return answer
```

9. Code Implementation

✓ Python

```
class Solution:

    def equalSubstring(
        self,
        s: str,
```



```
t: str,
maxCost: int
) -> int:

    left = 0
    currentCost = 0
    answer = 0

    for right in range(len(s)):

        currentCost += abs(
            ord(s[right]) -
            ord(t[right])
        )

        while currentCost > maxCost:

            currentCost -= abs(
                ord(s[left]) -
                ord(t[left])
            )

            left += 1

        answer = max(
            answer,
            right - left + 1
        )

    return answer
```

✓ Java

```
class Solution {

    public int equalSubstring(
        String s,
        String t,
        int maxCost
    ) {

        int left = 0;

        int currentCost = 0;

        int answer = 0;

        for (int right = 0;
            right < s.length();
```



```
        right++) {

            currentCost += Math.abs(
                s.charAt(right)
                - t.charAt(right)
            );

            while (
                currentCost > maxCost
            ) {

                currentCost -= Math.abs(
                    s.charAt(left)
                    - t.charAt(left)
                );

                left++;
            }

            answer = Math.max(
                answer,
                right - left + 1
            );
        }

        return answer;
    }
}
```

10. Time & Space Complexity

Time Complexity

Each pointer moves at most:

n times

Overall:

$O(n)$

Space Complexity

$O(1)$

11. Common Mistakes / Edge Cases

Mistake 1

Using nested loops.

Results in:

$O(n^2)$

**Mistake 2**

Not shrinking until window becomes valid.

Wrong:

if currentCost > maxCost:

Correct:

while currentCost > maxCost:

Mistake 3

Forgetting absolute difference.

Must use:

abs(...)

12. Detailed Dry Run (Step-by-Step Table)

Input:

s = "abcd"

t = "bcdf"

maxCost = 3

Cost Array:

[1,1,1,2]

Right	Cost Added	Window Cost	Left	Length
0	1	1	0	1
1	1	2	0	2
2	1	3	0	3
3	2	5	0	Invalid

Shrink:

Action	Cost
Remove 1	4
Remove 1	3

Now:

Window = [1,2]

Length = 2

Maximum length remains:

3



13. Common Use Cases (Real-Life / Interview)

Budget-Constrained Transformations

Modify data within a fixed budget.

Network Optimization

Convert packets while limiting transmission cost.

Image Processing

Apply transformations under resource limits.

String Similarity Problems

Measure longest affordable conversion.

14. Common Traps (Important!)

- Using if instead of while
- Forgetting absolute difference
- Recomputing costs repeatedly
- Not updating answer after shrinking

15. Builds To (Related LeetCode Problems)

1. LC 209 — Minimum Size Subarray Sum
2. LC 1004 — Max Consecutive Ones III
3. LC 1493 — Longest Subarray After Deleting One Element
4. LC 424 — Longest Repeating Character Replacement
5. LC 1208 — Get Equal Substrings Within Budget

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(n^2)$	$O(1)$	TLE
Prefix Sum + Binary Search	$O(n \log n)$	$O(n)$	Good
Sliding Window	$O(n)$	$O(1)$	Optimal

17. Why This Solution Works (Short Intuition)

The character conversion costs form a non-negative array. Finding the longest substring within a budget becomes equivalent to finding the longest subarray whose sum is at most `maxCost`, which is solved optimally using a sliding window.



18. Variations / Follow-Up Questions

Follow-Up 1

What if costs can be negative?

Sliding Window fails.

Use:

Prefix Sum + Monotonic Queue

Follow-Up 2

What if budget changes for multiple queries?

Use Prefix Sums + Binary Search.

Follow-Up 3

What if strings arrive as a stream?

Maintain a dynamic sliding window.